



U N I V E R S I D A D
Panamericana

UNIVERSIDAD PANAMERICANA

LABORATORIO IoT/DISEÑO ELECTRÓNICO

PRÁCTICA 1

ALUMNOS:

Carlo Gordillo Arenas, Alejandro Acosta Salazar y Eduardo Hernández Torres

OBJETIVO:

En esta práctica tenemos como objetivo establecer una comunicación inalámbrica bidireccional utilizando Bluetooth Low Energy (BLE) entre un microcontrolador ESP32 y una aplicación central desde el teléfono móvil, con el fin de que obtengamos lecturas variables simuladas y la escritura de los parámetros en Setpoint.

MARCO TEÓRICO:

BLE:

A diferencia del Bluetooth común, BLE está diseñado para transmitir pequeñas cantidades de información consumiendo energía en menor número. Quiere decir que trabaja con una banda de ISM de 2.4 GHz, lo que es ideal para aparatos que funcionan por medio de baterías, y la función de radio se emplea en un modo de reposo la mayor parte del tiempo, activándose solo para notificar cortas cantidades de datos.

Arquitectura GATT:

Arquitectura GATT es la forma en que se organizan y se transmiten los datos de BLE una vez que la conexión queda establecida. Se estructura en 3 pilares:

1. Servicio (UDD):

- Es una colección de datos y comportamientos relacionados, donde cada servicio se identifica de forma única desde un UDD.

2. Características:

- Aquí se contienen los datos reales, dándoles un valor y propiedad de definir lo que se puede hacer con esa información: lectura, notificaciones o escribir.

3. Descriptores:

- Son metadatos que expanden información de una sola característica, ya sea para definir unidades de medida o configurar sus notificaciones.

[Perfil GATT]

└─ [Servicio (UUID)]

└─ [Característica 1 (UUID)] ──> [Descriptor (Client Characteristic Configuration)]

└─ [Característica 2 (UUID)]

DIAGRAMA DE CONEXIONES:

No aplica.

DESARROLLO:

Primero, el código es el que da inicio al ESP32 como un servidor GATT y expone un servicio en dos características principales:

Librerías:

Con estos, los identificadores podrán generarse de forma aleatoria y asegurar que no se mezclen con otros servicios.

[SERVICE_UUID, CHAR_PRM_UUID, CHAR-SETPOINT-UUID]

Callbacks:

- Server Callbacks: Funcionan para que el cliente pueda volver al advertising si este se desconecta del estado de conexión de CABLE; así volverá a ser visible.

Setups:

- Se ejecutan por un orden: Inicialización, Servidores, Servicio, Características RPM, Características de Setpoints, Inicio.

Loops

- Cuando existe un dispositivo conectado, primero se simula desde el incremento de las RPM (+5.0 por ciclo) hasta llegar a 600. Por consiguiente, el valor flotando en una cadena de string, y usamos un sprint, así el valor de la setValue y lo envía activamente al cliente sin necesidad de mandar una Notify(). Finalmente, se añade un retraso de 500 ms para no saturar los canales de comunicación.

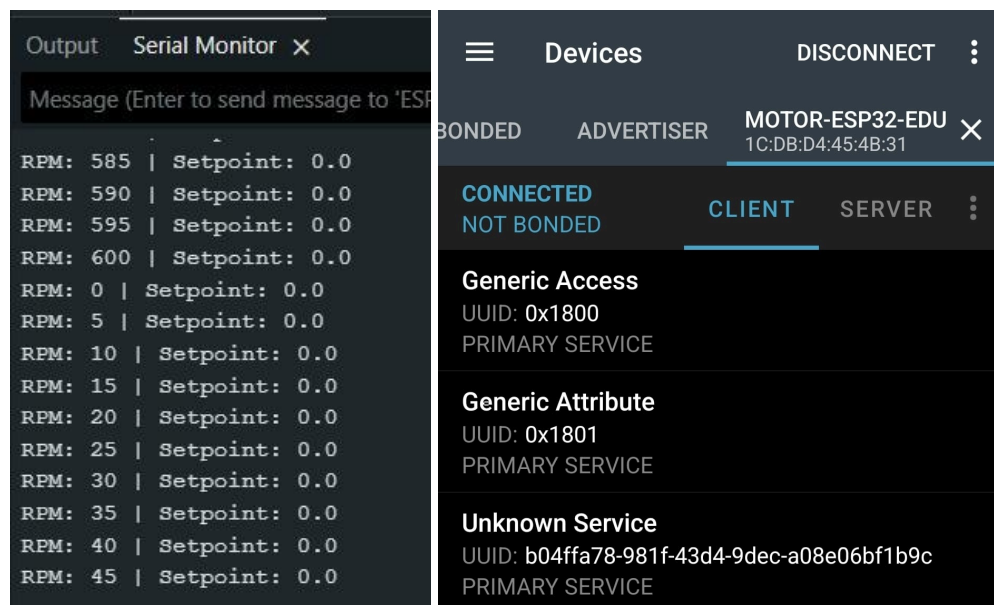
PROCEDIMIENTO DE PRUEBAS:

Primero se compiló el código totalmente armado en el IDE de Arduino y se envió al ESP32, para monitorearlo en serie, con el Monitor Serie en 115200 baudios, verificando el mensaje: BLE listo - esperando conexión... Una vez conectado en la app por BLE en el teléfono móvil, encontramos el dispositivo con nombre Motor-ESP32-EDU para terminar la conexión. Ahora en la app se presionó un símbolo con 3 flechas que activa las notificaciones, en la característica de RPM para observar cómo el valor cambiaba automáticamente en cada casi medio segundo. En la

aplicación, por último, envía un valor numérico a la característica de Setpoint y se verificó en el Monitor de serie del IDE de Arduino que el ESP32 lo recibiera correctamente.

RESULTADOS:

- El ESP32 anunció su presencia de manera correcta y fue visible inmediatamente en el escáner BLE.
- Cuando se conectó la aplicación, el monitor serie arrojó las siglas >>> Central conectado!
- Las notificaciones en RPM funcionaron y en la pantalla se mostró el incremento de 5 a 5.
- La recepción del setpoint se ejecutó bien.



[Serial Monitor y conexión en nRF Connect]

DISCUSIÓN Y ANÁLISIS DE RESULTADOS:

Por lo que se ve en las imágenes mostradas, se obtuvieron los resultados esperados, se pudo realizar la conexión sin problemas y ver los datos en el Serial Monitor.

CONCLUSIONES:

En la práctica encontramos formas de trabajar en la arquitectura GATT de BLE para interactuar de forma ágil con las variables de control; así aprendimos que una diferencia en cuándo enviamos datos de forma pasiva y cuándo usamos interrupciones debido a los callbacks por la recepción de los Setpoints, lo cual optimiza recursos de procesamiento en el ESP32, ya que encontramos que la correcta manera del ciclo de vida en el advertising es crítica para garantizar la disponibilidad y tamaño de un sistema embebido en un entorno real.